

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)B
- B)A and B
- C)Nothing
- D)A

Q2. What is the output?

```
def fun(x):
    return x * 2
```

```
a = fun
print(a(5))
```

- A)func(5)
- B)5
- C>Error
- D)10

Q3. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It prevents syntax errors
- B)It creates the main function
- C)It prevents all imports
- D)It identifies code intended to run when the file is executed directly

Q4. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

- A)24
- B)0
- C)10
- D)9

Q5. What will be printed?

```
def fun():
    print("A")
    return
    print("B")
```

```
fun()
```

- A)A B
- B)A
- C)B
- D)Nothing

Q6. What is the output?

```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)A
- B)A C
- C)B
- D)B C

Q7. What is the output?

```
try:
    print(10 / 0)
except Exception:
    print("A")
```



except ZeroDivisionError:

print("B")

A)SyntaxError

B)A

C)B

D)A B

Q8. What is the output?

def fun(x):

x += 10

return x

x = 5

y = fun(x)

print(x, y)

A)15 5

B)5 15

C)5 5

D)15 15

Q9. What happens when this code runs?

def first():

second()

def second():

third()

def third():

raise RuntimeError("Failure")

try:

first()

except RuntimeError:

print("Handled")

A)NameError

B)TypeError

C)Handled

D)RuntimeError cannot propagate through functions

Q10. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.modules

B)sys.path

C)sys.imports

D)__module__

Q11. What happens?

def fun(a, b):

return a + b

print(fun(10, a=20))

A)SyntaxError

B)TypeError

C)30

D)20

Q12. What will be printed?

def fun(a, b):

return a + b

print(fun(10, b=20))

A)TypeError

B)SyntaxError

C)20

D)30

Q13. What is the output?

def fun(a=10, b=20):

return a - b

print(fun(30))

A)20

B)30

C)-10

D)10

Q14. What happens here?

def fun(a):

return a

print(fun())

A)None

B)0

C)TypeError

D)NameError

Q15. What is the output?

def outer():

x = 10

def inner():

return x + 5



return inner

**f = outer()
print(f())**

- A)10
- B)NameError
- C)None
- D)15

Q16. What will be the output?

x = 10

**def fun():
 print(x)
 x = 20**

- fun()**
- A)20
 - B)NameError
 - C)10
 - D)UnboundLocalError

Q17. What will be printed?

**class InvalidAgeError(Exception):
 pass**

**def check_age(age):
 if age < 18:
 raise InvalidAgeError("Age is too low")
 return "Allowed"**

**try:
 print(check_age(15))
except InvalidAgeError as e:
 print(type(e).__name__)
 print(e)**

- A)Nothing, because custom exceptions cannot be caught
- B)InvalidAgeError
- 15
- C)Exception
- Age is too low
- D)InvalidAgeError
- Age is too low

Q18. What happens here?

**try:
 raise ValueError("Wrong value")
except ValueError as e:
 print(e)**

- A)e
- B)ValueError
- C)Nothing
- D)Wrong Value

Q19. What will normally happen?

**import math
import math**

- A)SyntaxError
- B)ImportError
- C)The module is normally loaded once and reused
- D)The module is normally loaded and executed twice

Q20. What will be the output?

**def fun(x):
 try:
 if x < 0:
 raise ValueError("negative")
 return x
 except ValueError:
 return 100
 finally:
 print("cleanup")**

print(fun(-5))

- A)cleanup
- 5
- B)negative
- cleanup
- C)cleanup
- 100
- D)100
- cleanup

Q21. What will be the output?

x = 10

**def outer():
 x = 20**



```
def inner():
    global x
    x = 30
```

```
inner()
print(x)
```

```
outer()
print(x)
```

A)30

30

B)30

20

C)20

30

D)20

10

Q22. What will happen?

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

A)SyntaxError

B)TypeError

C)25

D)15

Q23. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

A)10 only

B)10 followed by Finally

C)Finally followed by 10

D)Finally only

Q24. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

A)No Output

B)B

C)A, followed by propagation of the same exception

D)A, and execution ends normally

Q25. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)5.0

B)25

C)sqrt(25)

D)5

Q26. What will be printed?

```
def make_counter():
    count = 0
```

```
def counter():
    nonlocal count
    count += 1
    return count
```

```
return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)1

1

1

1

B)1

2

1

3

C)0

1

0

2

D)1

2

3

4



Q27. What will be printed?

try:

```
    print(10 / 0)
```

except ZeroDivisionError:

```
    print("A")
```

except Exception:

```
    print("B")
```

A)Nothing

B)A B

C)A

D)B

Q28. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

A)Both import only sqrt

B)There is no difference

C)First provides the module namespace; second imports sqrt directly

D)First imports only sqrt

Q29. What happens when this code executes?

```
def fun():
```

```
    print(variable)
```

```
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)NameError

B)10

C)UnboundLocalError

D)50

Q30. What will be printed?

try:

```
    x = int("10")
```

except ValueError:

```
    print("A")
```

else:

```
    print("B")
```

finally:

```
    print("C")
```

A)A

B)B C

C)B

D)A C

Q31. What will happen?

```
def fun(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    return n * fun(n)
```

```
print(fun(3))
```

A)3

B)RecursionError

C)TypeError

D)6

Q32. What does the following function return?

```
def fun():
```

```
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)None

B>Hello

C>Error

D>Hello None

Q33. What is the output?

```
def fun():
```

```
    try:
```

```
        print("A")
```

```
        raise ValueError
```

```
    except ValueError:
```

```
        print("B")
```

```
        return 10
```

```
    else:
```

```
        print("C")
```

```
        return 20
```

```
    finally:
```

```
        print("D")
```

```
print(fun())
```

A)A

B



10
D
B)A
C
D
20
C)A
B
D
10
D)A
B
C
D
10

Q34. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

A)[10]
[10]
B)[10, 10]
[10]
C)Error
D)[10]
[10, 10]

**Q35. Why is
try:**

```
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")  
codes's second handler never  
reached?
```

A)ZeroDivisionError is not an exception
B)ZeroDivisionError occurs before
Exception
C)Exception is a superclass of
ZeroDivisionError
D)except blocks execute randomly

Q36. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6
B)36
C)6.0
D)NameError

Q37. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

A)10
B)5
C)15
D)UnboundLocalError

Q38. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

A)[4, 8]
B)[2, 4]
C)[1, 3, 5]
D)[2, 4, 6, 8, 10]

**Q39. if test.py has
print("A")**

```
if __name__ == "__main__":
```



```
print("B")
```

If test.py is executed directly, what will be printed?

- A) Nothing
- B) B
- C) A followed by B
- D) A

Q40. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

- A) NameError
- B) It executes correctly
- C) SyntaxError
- D) TypeError

Q41. What will happen?

```
def outer():  
    x = 10  
  
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

```
outer()  
A) 20  
B) NameError  
C) SyntaxError  
D) 10
```

Q42. What will be printed?

```
x = 10  
  
def outer():  
    x = 20  
  
    def inner():  
        print(x)  
  
    x = 30  
    inner()
```

```
outer()  
A) 20  
B) 30
```

- C) 10
- D) NameError

Q43. What will be printed?

```
x = 10  
  
f = lambda: x  
  
x = 20
```

```
print(f())  
A) NameError  
B) 10  
C) None  
D) 20
```

Q44. Suppose module_a.py contains:

```
print("Module loaded")  
  
value = 10
```

And another file contains:

```
import module_a  
import module_a  
  
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A) Module loaded
- 10
- B) ImportError
- C) 10
- 10
- D) Module loaded
- Module loaded
- 10

Q45. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random  
A) Python automatically renames the local file  
B) random cannot be imported
```



C)The local file can shadow the intended standard-library module

D)Python will always reject it

Q46. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)5

B>Error

C)15

D)10

Q47. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:
```

```
    test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

A)Exception

B)MyError

C)TypeError

D)Something went wrong

Q48. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)6

B)3

C)5

D)None

Q49. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

```
print(fun())
```

A)RuntimeError

B)20

C)10

D)None

Q50. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A)x

display

B)Only Hello

C)10

Hello

D>Error

